

HiSkill: EMPOWERING LLM AGENTS WITH HIERARCHICAL SKILL GRAPHS

Yu Hao¹, Jinxuan Cai¹, Qi Zhang², Yawen Li¹, Zhiqiang Zhang³, Chuan Shi¹, Cheng Yang^{1†}

¹Beijing University of Posts and Telecommunications

²China Mobile Group Shaanxi Co., Ltd

³Ant Group

haoyuu@bupt.edu.cn, caijinxuan@bupt.edu.cn, zhangqi3@sn.chinamobile.com
warmly0716@126.com, lingyao.zzq@antfin.com, shichuan@bupt.edu.cn,
yangcheng@bupt.edu.cn

ABSTRACT

Skills have become an important abstraction for enabling large language model (LLM) agents to reuse past experience in long-horizon interactive tasks. However, existing trajectory-to-skill methods often produce flat collections of high-level textual skills that are stored and retrieved independently, leaving skill relations underutilized and maintaining a gap between high-level skills and executable actions. In this paper, we propose HiSkill, a hierarchical skill graph framework that organizes interaction trajectories into a directed graph with skill nodes, AtomicOp nodes, and typed edges. Specifically, the graph connects reusable high-level skills with executable action templates, while also capturing decomposition, temporal transition, compatibility, support, and recovery relations among them. At inference time, HiSkill retrieves a compact task-relevant subgraph and performs subgraph-guided task execution, where a symbolic task state, an active skill, and the retrieved subgraph guide the LLM agent to switch skills, select AtomicOps, and ground executable actions iteratively. Experiments on three interactive environments show that HiSkill outperforms state-of-the-art baselines while reducing inference token consumption, demonstrating the effectiveness of bridging high-level skills and executable action grounding through a hierarchical skill graph.

1 INTRODUCTION

Long-horizon interactive tasks require large language model (LLM) agents to solve multi-step problems through continuous interaction, where reusing historical experience has become an important way to improve future decision making (Zhao et al., 2024; Chhikara et al., 2025; Fang et al., 2025).

Skills have recently emerged as a key abstraction for organizing such reusable experience. A skill is commonly defined as an on-demand modular capability unit that provides procedural knowledge, such as description, instructions, or metadata to guide future decision making (Xu & Yan, 2026; Anthropic, 2025b; Liang et al., 2026). Compared with one-off prompts, skills allow LLM agents to avoid repeatedly solving similar tasks from scratch, thereby improving the efficiency and stability of multi-step task solving.

To reduce the cost of skill construction, recent methods propose to automatically extract skills from interaction trajectories. For example, SkillIRL distills noisy trajectories into a SkillBank and recursively evolves it through reinforcement learning (Xia et al., 2026a). D2Skill organizes reusable experience into skills for task guidance and error correction (Tu et al., 2026). CoEvoSkills automatically generates skill packages and iteratively refines them through a skill generator and a surrogate verifier (Zhang et al., 2026). The resulting skills are typically organized as flat collections of high-level textual descriptions.

However, existing trajectory-to-skill methods have two limitations. First, most generated skills remain coarse-grained strategies or textual guidance, leaving a gap between high-level skill descrip-

[†]Corresponding author.

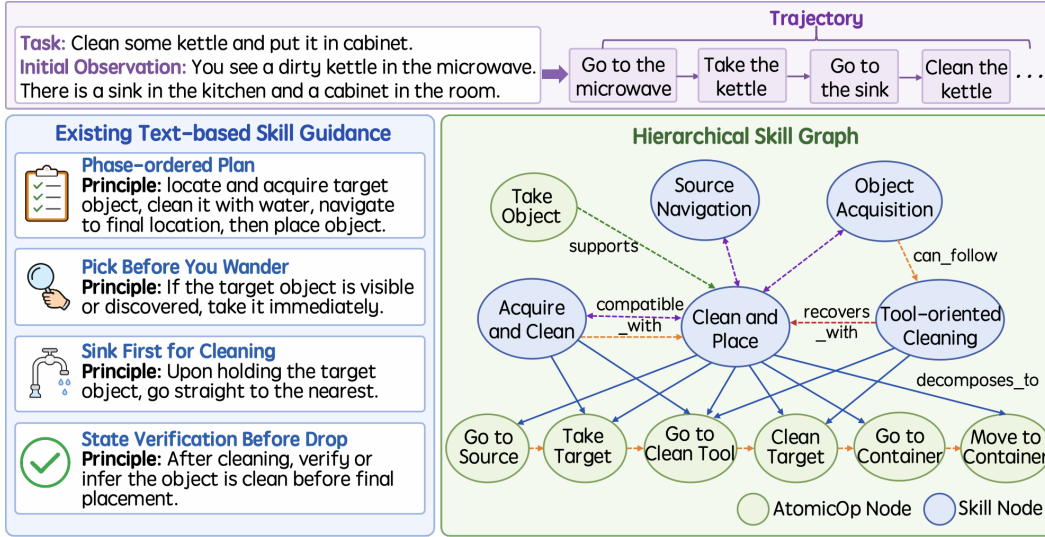


Figure 1: Existing methods distill trajectories into coarse-grained textual rules in a flat organization, while HiSkill builds a hierarchical graph to assist task execution, including high-level skill nodes, executable AtomicOp nodes and typed edges between them.

tions and concrete executable actions. A skill may describe what subgoal to achieve or what general procedure to follow, but actual interaction requires deciding the next concrete action, with arguments and execution timing. Simply injecting high-level skill text into the LLM context does not always allow the model to infer these details (Liu et al., 2026b; Li et al., 2026b). Second, many procedural steps are shared across tasks, but existing methods often construct and retrieve skills as independent units, underutilizing relations among skills and shared action patterns.

Recent graph-based methods highlight the importance of structure for skill use. For example, Graph of Skills retrieves dependency-aware skill bundles from an offline skill graph and shows that flat vector retrieval can miss execution dependencies (Liu et al., 2026a). GraSP compiles retrieved skills into typed directed acyclic graphs and emphasizes that the bottleneck has shifted from skill availability to skill orchestration (Xia et al., 2026b). Nevertheless, these methods typically construct graphs at the skill level, and fail to explore the diverse ways in which skills may relate during execution. As a result, relations among skills are not fully exploited, and the retrieved skills are mainly high-level textual guidance, leaving the connection to executable actions implicit.

To address these limitations, we propose **HiSkill**, a method that constructs hierarchical skill graphs from trajectories and uses them for interactive task solving. As shown in Figure 1, existing methods produce coarse-grained textual skills as high-level guidance, while HiSkill represents the trajectories as a hierarchical graph with skill nodes, AtomicOp nodes, and typed edges. Specifically, AtomicOp nodes denote executable action templates, while skill nodes denote reusable procedures composed of multiple AtomicOp nodes and enriched with structured metadata. The typed edges further capture procedural relations, allowing the graph to encode not only which procedures usually work, but also which skills can be composed, which auxiliary actions are needed, and which operations can be tried when execution fails.

At inference time, HiSkill retrieves a compact task-relevant subgraph through semantic-lexical matching and graph expansion. It then monitors a symbolic task state and an active skill, and uses them together with the retrieved subgraph and an LLM to switch or maintain the active skill, select an AtomicOp, or directly generate an action by an LLM. The resulting action is executed, and the new observation iteratively updates the task state for subsequent skill switching and AtomicOp selection. This design effectively bridges high-level skill abstraction and executable action grounding.

We summarize our main contributions as follows:

- We propose a hierarchical skill graph framework that converts trajectories into a directed graph with skill nodes, AtomicOp nodes and typed edges, bridging high-level skill abstraction and executable action templates.
- We design a pipeline that constructs hierarchical skill graphs, retrieves task-relevant sub-graphs, and performs subgraph-guided task execution.
- We compare HiSkill with state-of-the-art baselines across three interactive environments. HiSkill achieves average relative improvements of 17.33% in success rate, while reducing inference token consumption by 78.75% over the strongest baseline.

2 RELATED WORK

2.1 AGENTIC SKILLS

Agentic skills are reusable procedural capability units that package task-specific description, instructions, or metadata for LLM agents (Xu & Yan, 2026; Anthropic, 2025b; Lu et al., 2026). Early and deployed skill systems often rely on manual authoring, such as Anthropic Agent Skills organized through `SKILL.md` files (Anthropic, 2025a) and OpenAI Custom GPTs / GPT Actions built from custom instructions, knowledge files, and API schemas (OpenAI, 2024b). Manual skills are explicit and controllable, but scaling them to complex interactive environments requires substantial effort to specify procedures, applicability conditions, and execution details. To reduce this manual cost, recent methods automatically construct skills from trajectories and interaction experience. Voyager maintains an ever-growing executable code skill library in Minecraft (Wang et al., 2024); SkillRL and D2Skill distill trajectories into skill banks with different forms (Xia et al., 2026a; Tu et al., 2026); SkillX and CoEvoSkills study skill knowledge-base construction and automatic skill package generation (Wang et al., 2026; Zhang et al., 2026); and CUA-Skill represents GUI operation knowledge as parameterized skills for computer-use agents (Chen et al., 2026).

Beyond individual skill construction, recent work also studies skill evaluation, ecosystem organization, and structured use. SkillsBench evaluates the effectiveness of curated and self-generated skills (Li et al., 2026b); SkillNet and AgentSkillOS organize skills as large-scale assets with ontology, evaluation, and orchestration mechanisms (Liang et al., 2026; Li et al., 2026a); and Graph of Skills and GraSP introduce graph-structured retrieval or composition for large skill libraries (Liu et al., 2026a; Xia et al., 2026b). In addition, platforms such as SkillsMP, ClawHub, and LangSkills support skill packaging, publishing, browsing, and searching at scale, reflecting the shift from individual skills to ecosystem-level skill management (SkillsMP, 2026; OpenClaw, 2026; LabRAI, 2026). However, skills often remain at the level of coarse-grained strategies or textual guidance in existing systems. This leaves a gap between high-level skill descriptions and executable actions. Besides, the diverse ways in which skills may relate during execution are also underexplored.

2.2 EXPERIENCE AND MEMORY FOR LLM AGENTS

Experience and memory mechanisms allow LLM agents to reuse past interactions. For example, Reflexion stores verbal feedback for self-improvement (Shinn et al., 2023), ExpeL extracts natural-language insights from past tasks (Zhao et al., 2024), and Mem0 studies scalable long-term memory extraction and retrieval for agents (Chhikara et al., 2025). More closely related to task execution, MemP explores procedural memory by distilling past trajectories into step-level instructions and script-like abstractions (Fang et al., 2025).

Overall, memory-based methods demonstrate the value of reusing historical experience, but they typically store experience as reflections, text memories, summaries, or procedural instructions. They pay less attention to organizing high-level procedures, executable actions, and execution relations into a unified structure for runtime action grounding.

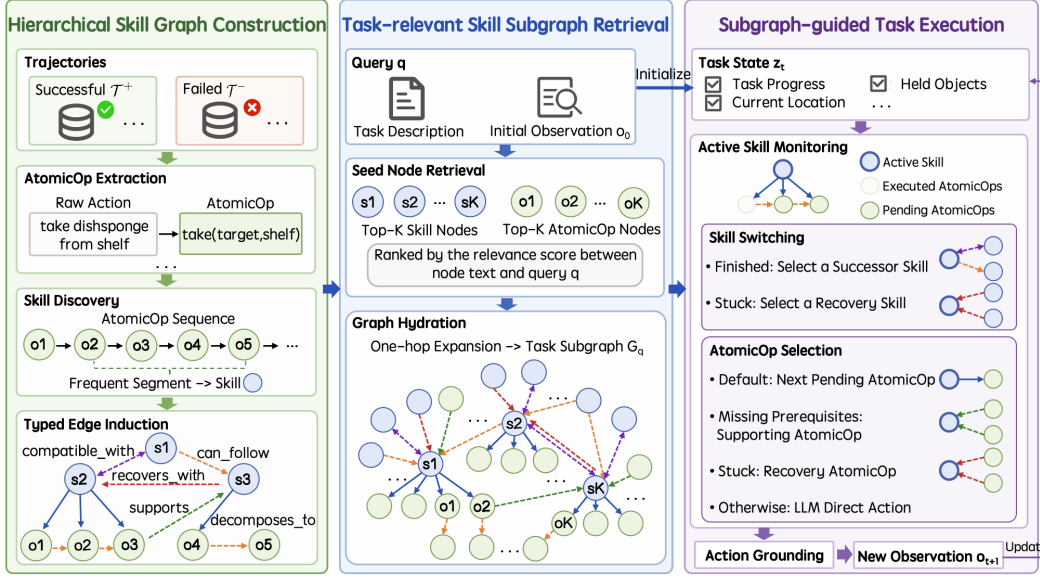


Figure 2: The overall framework of our proposed HiSkill.

3 METHODOLOGY

3.1 FRAMEWORK OVERVIEW

As shown in Figure 2, HiSkill consists of three main components: (1) *Hierarchical Skill Graph Construction* builds a directed graph from interaction trajectories, where AtomicOp nodes denote executable action templates, skill nodes denote high-level procedures composed of multiple AtomicOp nodes and structured metadata, and typed edges capture relations among them. (2) *Task-relevant Skill Subgraph Retrieval* retrieves a compact task-relevant subgraph from the constructed skill graph according to semantic similarity, lexical matching, and graph relations. (3) *Subgraph-guided Task Execution* monitors the current task state and active skill, and uses the retrieved subgraph as well as an LLM to switch skills or select AtomicOps for task execution.

3.2 HIERARCHICAL SKILL GRAPH CONSTRUCTION

We first construct a hierarchical skill graph from historical interaction trajectories. Formally, we partition the trajectory set $\mathcal{T}$ into:

$$\mathcal{T}^+ = \{\tau \in \mathcal{T} : r(\tau) = 1\}, \quad \mathcal{T}^- = \{\tau \in \mathcal{T} : r(\tau) = 0\}, \quad (1)$$

where $r(\tau) \in \{0, 1\}$ indicates whether τ successfully completes the task. Successful trajectories $\mathcal{T}^+$ provide reusable execution patterns, while failed trajectories $\mathcal{T}^-$ provide failure modes and recovery evidence. Now we will briefly introduce how to extract atomic operations, discover skills, and induce typed edges from these trajectories. Implementation details are provided in Appendix A.

3.2.1 ATOMICOP EXTRACTION.

We extract raw action sequences from interaction trajectories and canonicalize each raw action into an AtomicOp occurrence. Specifically, a schema-based canonicalizer parses the raw action, identifies its action type and arguments, replaces task-specific entities with placeholders, and produces a normalized action template. We then deduplicate and merge AtomicOp occurrences with the same normalized template, while accumulating their support counts across trajectories. The resulting AtomicOp node set V_O serves as the executable action vocabulary for later skill discovery and action grounding.

3.2.2 SKILL DISCOVERY.

We then mine skills from the AtomicOp sequences of successful trajectories. Specifically, we identify frequent action patterns, and abstract each retained pattern into a skill node, forming the skill set V_S . Each skill stores an ordered AtomicOp sequence and structured metadata, including applicability states, expected state changes, representative examples, argument candidates, support count, and failure hints. Therefore, a skill is not merely a textual hint, but a reusable procedural abstraction grounded in observed action patterns. To improve readability, we use an LLM only to refine natural-language fields such as names and descriptions, without changing the mined AtomicOp structure.

Typed Edge Induction. Finally, we induce typed edges from successful and failed trajectories to construct a hierarchical skill graph:

$$G = (V, E), \quad V = V_S \cup V_O, \quad E \subseteq V \times V \times \mathcal{L}, \quad (2)$$

where V_S and V_O denote the sets of skill and AtomicOp nodes, and each edge $(u, v, \ell) \in E$ denotes a directed relation of type ℓ from node u to node v . The relation type set is:

$$\mathcal{L} = \{\text{decomposes_to}, \text{can_follow}, \text{compatible_with}, \text{supports}, \text{recovers_with}\}. \quad (3)$$

These edges are induced from trajectory statistics, state compatibility, failure patterns, and semantic matching. Specifically, `decomposes_to` edges link each skill node to the ordered AtomicOp nodes, specifying how a skill is decomposed into executable action templates; `can_follow` edges connect skill-to-skill or AtomicOp-to-AtomicOp nodes, and are induced by counting ordered co-occurrences in successful skill or AtomicOp sequences, capturing directed temporal continuations; `compatible_with` edges connect skill nodes that frequently co-occur in successful trajectories and have compatible pre/post-states, capturing unordered skill composition as symmetric directed edges; `supports` edges link auxiliary AtomicOp nodes to skill nodes when the AtomicOp frequently appears in the context of a skill and helps satisfy missing prerequisites or provides supporting state evidence; `recovers_with` edges connect recovery skill/AtomicOp nodes to failed or stalled skill/AtomicOp nodes, and are inferred from failed trajectories, repeated ineffective actions, failure hints, missing-state evidence, and semantic/state matching.

In summary, successful trajectories mainly induce decomposition, transition, compatibility, and support relations, while failed trajectories mainly induce recovery relations. The resulting hierarchical skill graph encodes high-level skills, executable AtomicOps, and their relations, providing a structured foundation to assist task execution.

3.3 TASK-RELEVANT SKILL SUBGRAPH RETRIEVAL

After constructing the hierarchical skill graph $G = (V, E)$, we retrieve a compact task-relevant subgraph for each task q instead of using the full graph:

$$G_q = (V_q, E_q), \quad V_q \subseteq V, \quad E_q \subseteq E. \quad (4)$$

The retrieval process first selects task-relevant skill and AtomicOp seed nodes, and then enriches them with graph relations needed for execution. Implementation details are provided in Appendix B.

3.3.1 SEED NODE RETRIEVAL.

We construct a retrieval query x_q by concatenating the task description and initial observation. For each node $v \in V$, we compute a relevance score using its textualized representation d_v . Following the hybrid retrieval strategy used in ToolScope (Liu et al., 2026d) and GoS (Liu et al., 2026a), we combine dense semantic similarity and sparse lexical matching:

$$\text{score}(v \mid q) = \lambda \text{Dense}(x_q, d_v) + (1 - \lambda) \text{Sparse}(x_q, d_v), \quad (5)$$

where $\text{Dense}(\cdot)$ computes cosine similarity between query and node embeddings, $\text{Sparse}(\cdot)$ is BM25 (Robertson & Zaragoza, 2009) score based on lexical term overlap, and $\lambda \in [0, 1]$ balances the two signals. We rank skill and AtomicOp nodes separately according to Eq. (5), and retain the top- K nodes from each type, and denote their union as V_{seed} .

3.3.2 GRAPH HYDRATION.

After obtaining the seed nodes, we enrich them with one-hop expansion to form the task-relevant subgraph. Considering the directionality of different edge types, the expanded node set is defined as:

$$V_q = V_{\text{seed}} \cup \{v \mid (u, v, \ell) \in E, u \in V_{\text{seed}}, \ell = \text{decomposes_to}\} \cup \{v \mid (v, u, \ell) \in E, u \in V_{\text{seed}}, \ell \neq \text{decomposes_to}\}. \quad (6)$$

Then we retain all typed edges among the selected nodes $E_q = \{(u, v, \ell) \in E \mid u \in V_q, v \in V_q\}$. The resulting subgraph contains task-relevant skill nodes, AtomicOp nodes, and their typed relations for runtime execution.

3.4 SUBGRAPH-GUIDED TASK EXECUTION

The retrieved subgraph provides candidate skill and AtomicOp nodes for assisting task execution. At runtime, the system performs stepwise task execution by monitoring the current task state and active skill: the task state records the current interaction status, and the active skill serves as the current execution anchor. At each step, the system updates the state, switches or keeps the active skill, selects a relevant AtomicOp target, and grounds it into a concrete executable action. Implementation details are provided in Appendix C.

3.4.1 TASK STATE.

At step t , the system maintains a task state z_t , implemented as a symbolic key-value memory. After executing a concrete action a_t and receiving the next observation o_{t+1} , the state is updated as:

$$z_{t+1} = \text{UpdateState}(z_t, a_t, o_{t+1}). \quad (7)$$

The state records execution-relevant evidence, such as task progress, held objects, current location, and visited entities. It is later used to check skill completion and applicability, detect missing prerequisites, and identify stagnation or repeated failures.

3.4.2 SKILL SWITCHING.

At step t , the system maintains an active skill node $s_t \in V_q \cap V_S$. The active skill node is associated with an ordered sequence of AtomicOp nodes through its `decomposes_to` edges. The initial skill node is selected from V_q by the LLM according to the task, initial observation, and task state. The active skill will be switched in two cases: when the active skill is finished, a successor skill node is selected from V_q through `can_follow` or `compatible_with` edges; when execution is stuck or repeatedly fails, the system switches to a recovery skill node through `recovers_with` edges.

3.4.3 ATOMICOP SELECTION.

By default, it selects the next unexecuted AtomicOp node from the active skill through `decomposes_to` edges; when required prerequisites are missing, it selects a support AtomicOp node through `supports` edges; when execution is stuck or repeatedly fails, it selects a recovery AtomicOp node through `recovers_with` edges; if no subgraph-related action is suitable, the system allows an LLM direct-action option. We denote the selected operation target as u_t , which can be either an AtomicOp node or an LLM-generated action.

3.4.4 ACTION GROUNDING AND STATE UPDATE.

Given the selected operation target u_t , the system grounds it into a concrete executable action according to the current observation and task state:

$$a_t = \text{Grounding}(u_t, o_t, z_t), \quad (8)$$

where a_t denotes the executable action with instantiated arguments.

After executing a_t , the system receives a new observation o_{t+1} and updates the task state according to Eq. (7). The updated state is then used for subsequent skill switching and AtomicOp selection. This iterative process continues until the task is completed.

Method	ALFWorld													
	Seen							Unseen						
	Pick	Look	Clean	Heat	Cool	Pick2	All	Pick	Look	Clean	Heat	Cool	Pick2	All
<i>Prompt-based Methods</i>														
ReAct	82.86	84.62	59.26	0.00	24.00	87.50	59.29	87.50	100.00	<u>93.55</u>	8.70	57.14	64.71	66.42
Reflexion	85.71	100.00	62.96	68.75	84.00	58.33	75.71	<u>95.83</u>	83.33	83.87	43.48	90.48	82.35	79.85
<i>Memory-based Methods</i>														
ExpeL	77.14	100.00	70.37	6.25	48.00	41.67	58.57	83.33	66.67	80.65	30.43	66.67	82.35	68.66
Mem0	<u>94.29</u>	92.31	77.78	37.50	76.00	87.50	80.00	91.67	94.44	<u>93.55</u>	39.13	85.71	100.00	83.58
MemP	<u>88.57</u>	100.00	88.89	75.00	92.00	83.33	87.86	<u>95.83</u>	100.00	100.00	<u>60.87</u>	95.24	70.59	88.06
SimpleMem	86.11	86.67	62.96	0.00	20.00	82.61	60.71	92.00	83.33	83.87	4.55	33.33	76.47	63.43
<i>Skill-based Methods</i>														
Vector Skills	85.71	76.92	66.67	6.25	52.00	<u>91.67</u>	67.14	<u>95.83</u>	88.89	70.97	4.35	47.62	64.71	61.94
SkillNet	91.43	61.54	66.67	68.75	80.00	75.00	76.43	<u>95.83</u>	83.33	83.87	47.83	76.19	76.47	77.61
GoS	88.57	92.31	66.67	6.25	24.00	83.33	62.86	<u>95.83</u>	94.44	<u>93.55</u>	4.35	38.10	58.82	65.67
HiSkill	100.00	100.00	92.59	75.00	96.00	95.83	94.29	100.00	100.00	87.10	69.57	100.00	100.00	91.79

Table 1: Performance on ALFWorld with Gemini-2.5-Pro: Success rate (%) is reported. The best results are highlighted in boldface, and the second-best results are underlined.

Method	WebShop		ScienceWorld			
	Score	Succ.	Seen		Unseen	
			Score	Succ.	Score	Succ.
<i>Prompt-based Methods</i>						
ReAct	51.46	34.40	39.36	34.54	39.65	38.86
Reflexion	35.88	29.20	<u>66.74</u>	63.92	58.96	52.61
<i>Memory-based Methods</i>						
ExpeL	<u>69.73</u>	<u>55.40</u>	63.73	63.92	58.23	59.24
Mem0	46.00	29.20	56.48	51.55	52.84	50.71
MemP	53.33	39.20	62.91	<u>64.43</u>	<u>67.20</u>	<u>61.14</u>
SimpleMem	50.13	32.00	64.05	61.86	57.95	52.13
<i>Skill-based Methods</i>						
Vector Skills	51.84	36.00	30.13	26.80	35.25	27.49
SkillNet	40.57	35.60	60.82	58.69	59.11	56.40
GoS	52.15	36.80	36.81	38.14	39.72	41.71
HiSkill	77.50	67.60	88.72	84.02	84.20	81.04

Table 2: Performance on WebShop and ScienceWorld with Gemini-2.5-Pro: Average score and success rate (%) are reported. The best results are highlighted in boldface, and the second-best results are underlined.

4 EXPERIMENTS

We conduct extensive experiments to answer the following research questions (RQs): **RQ1:** How does the proposed HiSkill perform compared with state-of-the-art baselines? **RQ2:** How does HiSkill perform in terms of inference token consumption? **RQ3:** How do the key components contribute to the overall effectiveness? **RQ4:** How sensitive is HiSkill to key hyperparameters? Besides, we provide a case study in Appendix E.3.

4.1 EXPERIMENTAL SETUP

4.1.1 DATASETS.

We evaluate our method on three interactive environments: ALFWorld (Shridhar et al., 2021), WebShop (Yao et al., 2022), and ScienceWorld (Wang et al., 2022). These environments cover embodied household manipulation, web shopping, and scientific experimentation, respectively, and require multi-step decision making under textual observations. Detailed dataset descriptions and statistics are provided in Appendix D.

4.1.2 EVALUATION.

For ALFWorld and WebShop, we follow the evaluation settings used in GiGPO (Feng et al., 2025) and SkillRL (Xia et al., 2026a). For ScienceWorld, we follow ETO (Song et al., 2024) and SkillNet (Liang et al., 2026). ALFWorld and ScienceWorld are evaluated on both Seen and Unseen splits. We further hold out 10% of the training set as a validation split for hyperparameter tuning and use the remaining training data for skill graph construction. Since ALFWorld provides a binary task completion signal, we report success rate (%) for each subtask and the overall average. For WebShop and ScienceWorld, which provide dense task scores, we report both average score and success rate (%). All experiments are conducted three times, and the results are reported as the average value.

4.1.3 BACKBONE LLMs.

We report the main results with two backbone LLMs: Gemini-2.5-Pro (Comanici et al., 2025) and GPT-5.2-Codex (OpenAI, 2025). All additional experiments and analyses are conducted under Gemini-2.5-Pro.

4.1.4 BASELINES.

For a fair comparison, we evaluate HiSkill against training-free methods from three categories. **Prompt-based methods** include **ReAct**, which interleaves reasoning, actions, and observations for step-by-step task solving (Yao et al., 2023), and **Reflexion**, which converts task feedback into verbal reflections for self-improvement (Shinn et al., 2023). **Memory-based methods** include **ExpeL**, which extracts natural-language insights from past tasks and retrieves them during inference (Zhao et al., 2024); **Mem0**, which maintains a long-term experience pool to guide action selection (Chhikara et al., 2025); **MemP**, which distills historical trajectories into procedural memory for decision making (Fang et al., 2025); and **SimpleMem**, which manages hierarchical memory for long-term information retrieval (Liu et al., 2026c). **Skill-based methods** include **Vector Skills**, which performs flat semantic retrieval over skills; **SkillNet**, which builds a multi-task skill library for task-specific skill retrieval (Liang et al., 2026); and **GoS**, which retrieves dependency-aware skill bundles from an offline skill graph using semantic-lexical seeding and Personalized PageRank (Liu et al., 2026a).

4.1.5 IMPLEMENTATION DETAILS.

The implementation details of HiSkill are provided in Appendix A–C. Besides, we tune two hyperparameters on the validation set: the dense–sparse retrieval weight λ over $\{0, 0.25, 0.50, 0.75, 1.00\}$ and the seed node budget K over $\{2, 4, 6, 8, 10\}$. Both analyses are reported in the Hyperparameter Analysis section.

4.2 MAIN RESULTS (RQ1)

Tables 1 and 2 report the results of our method and the baselines on three interactive environments with Gemini-2.5-Pro. The counterpart results with GPT-5.2-Codex are provided in Appendix E.1. Overall, our method achieves the best overall performance on all datasets. Compared with the strongest baseline across all datasets and splits, our method achieves an average relative improvement of 17.33% in success rate and 22.95% in score. On ALFWorld, it obtains the best or tied-best results on 5 out of 6 subtasks, indicating stable advantages across different task types.

Method	ALFWorld		WebShop	ScienceWorld	
	Seen	Unseen		Seen	Unseen
<i>Prompt-based Methods</i>					
ReAct	27,324	28,315	133,164	32,140	30,055
Reflexion	48,486	43,180	363,789	<u>22,752</u>	25,678
<i>Memory-based Methods</i>					
ExpeL	15,173	19,021	76,505	24,246	25,024
Mem0	50,555	52,374	131,969	60,560	51,605
MemP	32,476	34,110	156,291	53,224	60,297
SimpleMem	26,053	27,880	135,448	60,571	81,527
<i>Skill-based Methods</i>					
Vector Skills	52,973	62,280	139,953	42,101	43,485
SkillNet	66,053	58,992	77,731	295,143	299,667
GoS	58,118	59,407	138,240	41,006	46,867
HiSkill	6,462	6,838	4,501	7,747	8,135

Table 3: Average token consumption per task. Lower is better. The best results are highlighted in boldface, and the second-best results are underlined.

4.2.1 COMPARED WITH PROMPT-BASED METHODS.

ReAct and Reflexion rely mainly on in-context reasoning, actions, and feedback for step-by-step decision making. Although they perform well on some subtasks, they do not explicitly reuse historical interaction experience, leading to less stable performance. By contrast, our method abstracts reusable procedures from historical trajectories into a directed graph. On the ScienceWorld Seen split, compared with the strongest baseline Reflexion, our method improves score and success rate by 32.93% and 31.45%, respectively.

4.2.2 COMPARED WITH MEMORY-BASED METHODS.

ExpeL, Mem0, MemP, and SimpleMem enhance decision making with historical experience, external memory, or procedural memory. However, their experience is typically stored as textual memories, summaries, or trajectory fragments, without explicitly organizing high-level procedures, executable actions, and execution relations. Compared with the strongest baselines, our method improves ALFWorld Seen and Unseen overall success rates over MemP by 7.32% and 4.24%, improves WebShop score and success rate over ExpeL by 11.14% and 22.02%, and improves ScienceWorld Unseen score and success rate over MemP by 25.30% and 32.55%.

4.2.3 COMPARED WITH SKILL-BASED METHODS.

Vector Skills, SkillNet, and GoS all exploit skill information, but they remain limited by flat semantic retrieval, skill-library selection, or skill-level dependency completion. These methods mainly use skills as high-level strategies or textual guidance, leaving a gap to executable actions. Our method jointly models skill nodes, AtomicOp nodes, and typed relations, and further uses task state for runtime decision making, which leads to consistently stronger performance than existing skill-based baselines across all three datasets.

4.3 TOKEN CONSUMPTION ANALYSIS (RQ2)

Table 3 compares the average token consumption per task. Our method consistently achieves the lowest token usage across all datasets and splits. Compared with the lowest-token baseline on each setting, HiSkill reduces token consumption by 61.10%, 94.12%, and 66.76% on ALFWorld, WebShop, and ScienceWorld, respectively.

This indicates that the performance gains of HiSkill do not come from providing the LLM with longer trajectories, larger memory contents, or extensive skill texts. Instead, the retrieved task-relevant subgraph provides a compact representation of reusable experience, enabling the LLM to

Method	ALFWorld		WebShop	ScienceWorld	
	Seen	Unseen		Seen	Unseen
w/o Atom.	87.14	86.57	62.20	77.32	78.20
w/o Edge	84.29	84.33	60.80	74.23	73.46
w/o Sup./Rec.	<u>87.86</u>	<u>89.55</u>	<u>63.80</u>	78.35	74.88
w/o State	86.42	85.82	62.60	<u>79.38</u>	<u>78.67</u>
Static Subgraph	75.00	79.10	53.00	69.59	68.72
Top- K Prompt	66.43	78.36	40.80	57.73	47.39
GoS	62.86	65.67	36.80	38.14	41.71
OurG+GoSUse	80.00	82.86	53.20	51.03	54.50
GoSG+OurUse	72.86	75.71	51.80	56.70	51.18
Full	94.29	91.79	67.60	84.02	81.04

Table 4: Ablation study results measured by success rate (%). The best results are highlighted in boldface, and the second-best results are underlined.

reason over structured skills and executable actions with substantially reduced context, demonstrating that hierarchical skill graphs enable more efficient experience utilization during long-horizon task execution.

4.4 ABLATION STUDIES (RQ3)

Table 4 reports the success rates of different ablation variants. Overall, the full method achieves the best results across all datasets and splits. Compared with the strongest ablated variant, it obtains relative improvements of 7.32%, 2.50%, 5.96%, 5.85%, and 3.01% on ALFWorld Seen, ALFWorld Unseen, WebShop, ScienceWorld Seen, and ScienceWorld Unseen, respectively. The variants are analyzed as follows.

4.4.1 GRAPH REPRESENTATION.

w/o Atom. removes AtomicOp nodes as independent nodes for retrieval and dispatch, while **w/o Edge** removes all typed edges. Both variants degrade performance, indicating that high-level skills alone are insufficient for fine-grained interactive execution. AtomicOps bridge the gap between skill descriptions and executable actions, while typed edges make the retrieved subgraph a usable procedural context rather than a set of isolated nodes.

4.4.2 RUNTIME MECHANISMS.

w/o Sup./Rec. removes support and recovery edges and the corresponding inference-time operations, while **w/o State** removes the task state. Both variants underperform the full method, showing that support/recovery candidates are important for handling missing prerequisites and stalled execution, while the task state provides the runtime evidence required for reliable skill switching and AtomicOp selection.

4.4.3 SUBGRAPH USAGE.

Static Subgraph directly uses the retrieved subgraph as a fixed execution structure without using the LLM to dynamically select skills or AtomicOps, while **Top- K Prompt** only injects the retrieved top- K skill texts into the prompt. Their lower performance suggests that the retrieved subgraph is not a fixed execution plan, and that high-level skill text alone does not reliably help the LLM infer concrete steps, action arguments, or execution timing.

4.4.4 COMPONENT TRANSFER.

OurG+GoSUse uses our graph representation with the GoS-style retrieval and skill-bundle usage strategy, while **GoSG+OurUse** uses the GoS graph with our subgraph retrieval and task execution mechanism. OurG+GoSUse clearly outperforms vanilla GoS, showing that our graph representation

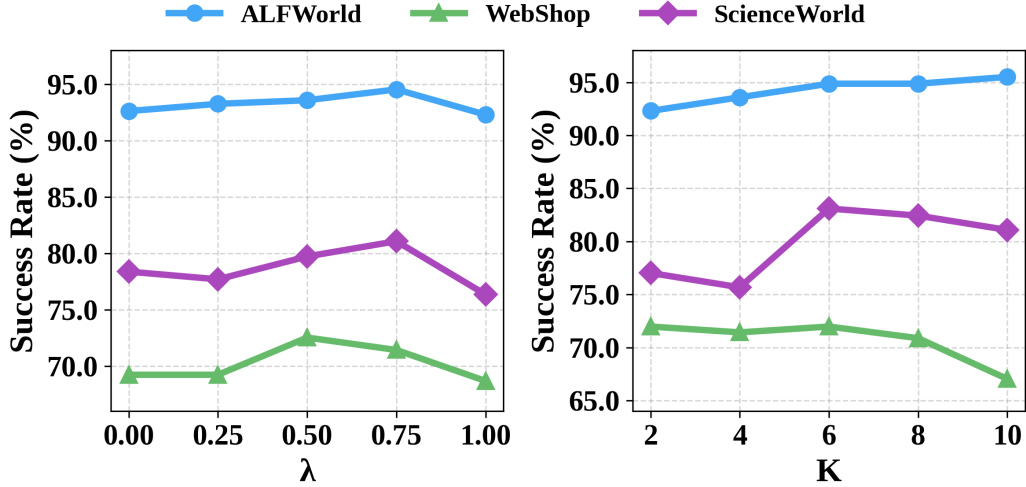


Figure 3: Validation sensitivity to the retrieval weight λ and seed node budget K measured by success rate.

provides more useful hierarchical and typed structure. GoSG+OurUse also improves over GoS, indicating that our execution mechanism can better use an existing skill graph. However, both transfer variants remain below the full method, suggesting that graph representation and runtime usage should be designed jointly.

4.5 HYPERPARAMETER ANALYSIS (RQ4)

We analyze two hyperparameters on the validation set: the dense-sparse retrieval weight λ and the seed node budget K . Figure 3 shows the results under different hyperparameters.

For λ , intermediate values generally outperform using only dense or sparse retrieval, indicating that semantic similarity and lexical matching provide complementary signals. ALFWorld is relatively insensitive to λ , while WebShop performs best at $\lambda = 0.5$ and ScienceWorld at $\lambda = 0.75$. Therefore, we select the best λ for each dataset based on validation performance and keep it fixed in the main experiments.

For the seed node budget K , increasing K can provide more candidate skill and AtomicOp nodes, but also introduces more context and decision noise. The results show that $K = 6$ provides a strong overall trade-off: it achieves the best or near-best success rate on the three datasets, while avoiding the larger context cost of higher budgets. Although larger K slightly improves ALFWorld, it degrades WebShop and ScienceWorld. We therefore set $K = 6$ in the main experiments. Score-based sensitivity results for WebShop and ScienceWorld are provided in Appendix E.2.

5 CONCLUSION

In this paper, we introduce HiSkill, a hierarchical skill graph framework for augmenting LLM agents in interactive tasks. HiSkill organizes historical interaction trajectories into a directed graph with skill nodes, AtomicOp nodes, and typed edges. At inference time, HiSkill retrieves a task-relevant subgraph and then monitors a task state and an active skill. Together with the subgraph and an LLM, it performs skill switching, AtomicOp selection, and action grounding. Experimental results show that HiSkill consistently improves task performance while substantially reducing inference token consumption across interactive environments. Future work will explore online skill graph evolution, adaptive skill refinement, and broader applications to more open-ended interactive scenarios.

REFERENCES

- Anthropic. Agent skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2025a. Claude API Documentation.
- Anthropic. Equipping agents for the real world with agent skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2025b. Anthropic Engineering Blog.
- Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, et al. Cua-skill: Develop skills for computer using agent. *arXiv preprint arXiv:2601.21123*, 2026.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. In *ECAI 2025 – 28th European Conference on Artificial Intelligence*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pp. 2993–3000, 2025.
- Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. *Advances in Neural Information Processing Systems*, 38:46375–46408, 2025.
- LabRAI. Langskills. <https://github.com/LabRAI/LangSkills>, 2026. Accessed 2026-04-01.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026a.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026b.
- Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, et al. Skillnet: Create, evaluate, and connect ai skills. *arXiv preprint arXiv:2603.04448*, 2026.
- Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. Graph-of-skills: Dependency-aware structural retrieval for massive agent skills. *arXiv preprint arXiv:2604.05333*, 2026a.
- Hongjun Liu, Yifei Ming, Shafiq Joty, and Chen Zhao. Harnessing llm agents with skill programs. *arXiv preprint arXiv:2605.17734*, 2026b.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026c.
- Marianne Menglin Liu, Daniel Garcia, Fjona Parllaku, Vikas Upadhyay, Fahad Shah, and Dan Roth. Toolscope: Enhancing llm agent tool use through tool merging and context-aware filtering. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 34095–34119, 2026d.

- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026.
- OpenAI. text-embedding-3-large. <https://developers.openai.com/api/docs/models/text-embedding-3-large>, 2024a. Official model documentation, accessed 2026-04-01.
- OpenAI. Gpt actions. <https://developers.openai.com/api/docs/actions/introduction>, 2024b. OpenAI Developers Documentation.
- OpenAI. Introducing gpt-5.2-codex. <https://openai.com/index/introducing-gpt-5-2-codex>, 2025. Official model announcement.
- OpenClaw. Clawhub registry. <https://openclawdoc.com/docs/skills/clawhub>, 2026. Accessed 2026-04-01.
- Stephen Robertson and Hugo Zaragoza. *The probabilistic relevance framework: BM25 and beyond*, volume 4. Now Publishers Inc, 2009.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- SkillsMP. Agent skills marketplace. <https://skillsmp.com>, 2026. Accessed 2026-04-01.
- Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. Trial and error: Exploration-based trajectory optimization of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7584–7600, 2024.
- Songjun Tu, Chengdong Xu, Qichao Zhang, Yaocheng Zhang, Xiangyuan Lan, Linjing Li, and Dongbin Zhao. Dynamic dual-granularity skill bank for agentic rl. *arXiv preprint arXiv:2603.28716*, 2026.
- Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, et al. Skillx: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*, 2026.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. Scienceworld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11279–11298, 2022.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. In *ICLR 2026 Workshop on Recursive Self-Improvement (RSI)*, 2026a.
- Tianle Xia, Lingxiang Hu, Yiding Sun, Ming Xu, Lan Xu, Siying Wang, Wei Xu, and Jie Jiang. Grasp: Graph-structured skill compositions for llm agents. *arXiv preprint arXiv:2604.17870*, 2026b.
- Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, et al. Coevoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024.

Appendix

A DETAILS OF HIERARCHICAL SKILL GRAPH CONSTRUCTION

This appendix provides implementation details for trajectory processing, AtomicOp extraction, skill discovery, and typed edge induction.

A.1 TRAJECTORY PROCESSING

The system interacts with environments and collects task-solving trajectories, where each trajectory contains the task description, observations, raw actions, environment feedback, and the final task outcome. We store each trajectory as:

$$\{\text{task_id}, \text{env_name}, \text{task_description}, \text{task_type}, \text{steps}, \text{success}, \text{metadata}\}. \quad (9)$$

Here, `steps` contains the interaction history, including observations, raw actions, and feedback, while `success` indicates whether the task is completed. This unified format allows trajectories from different environments to be processed by the same construction pipeline.

A.2 ATOMICOP EXTRACTION

Raw actions often have environment-specific surface forms and contain task-specific entities. Each raw action is parsed into an action type and structured arguments. Task-specific entities and constraints identified from the task description are replaced with semantic placeholders, while reusable literal values are retained. The normalized template is serialized as `action_type | key = value | ...`, with argument keys sorted to ensure a stable representation. Occurrences with the same normalized template are deduplicated and merged into one AtomicOp node. Each AtomicOp keeps the raw action for traceability, the normalized template for reuse, placeholder arguments for instantiation, and support statistics for later retrieval and graph construction. Figure 4 shows representative canonicalization examples.

A.3 SKILL DISCOVERY

We convert successful trajectories into AtomicOp sequences and enumerate recurring AtomicOp subsequences. A candidate subsequence is retained only if it appears in at least two successful trajectories and contains at least two AtomicOps. Duplicate or heavily overlapping candidates are further filtered to keep compact and distinct patterns.

For each retained pattern, we select a representative normalized AtomicOp sequence and abstract it into a skill node. When multiple candidate sequences are available, we prefer the one that is more general and easier to instantiate, particularly one that better preserves task-level placeholders such as target objects, target receptacles, tools, or product constraints. The selected sequence defines the executable procedure of the skill. From its supporting trajectories, we derive structured metadata, including pre-states, post-states, argument candidates, representative examples, support count, and failure hints. For each argument field, at most six candidate values are retained to keep the skill representation compact.

Figure 5 presents representative skill records, showing their AtomicOp sequence, pre-states, post-states, examples, failure hints, and support count. Additional metadata, such as complete argument candidates, constraint tags, and detailed failure descriptions, is retained in the skill graph but omitted from the figure for readability.

Skill Text Refinement. After a skill is discovered from recurring AtomicOp patterns, we use an LLM only to refine its natural-language fields for readability and retrieval. The refinement prompt rewrites the skill name, description, and failure hints, while keeping the AtomicOp sequence and structured metadata unchanged. This ensures that the skill structure remains grounded in trajectory evidence, and the LLM only improves the textual representation. Figure 6 shows the prompt template.

AtomicOp Canonicalization Examples	
go.to	Raw Action: go to cart Normalized Template: go.to location={target.receptacle} Arguments: {"location": "{target.receptacle}"} Support: 3436
move	Raw Action: move dish sponge to cart Normalized Template: move object={target.object} destination={target.receptacle} Arguments: {"object": "{target.object}", "destination": "{target.receptacle}"} Support: 3394
take	Raw Action: take toilet paper from countertop Normalized Template: take object={target.object} source=countertop Arguments: {"object": "{target.object}", "source": "countertop"} Support: 235

Figure 4: Examples of AtomicOp canonicalization.

Skill Representation Examples	
Pick Two Objects and Place	AtomicOp Sequence: go.to → take → go.to → move → go.to → take → go.to → move Pre-states: holding.target, target.located, navigation.needed Post-states: location.progress, target.placed Example: put two toilet paper in sidetable. Failure Hint: Loop after only one target is found. Support: 52
Clean and Place	AtomicOp Sequence: go.to → take → go.to → clean → go.to → move Pre-states: target.located, tool.ready, navigation.needed Post-states: process.progress, target.placed Example: clean some kettle and put it in cabinet. Failure Hint: Repeat checked locations or miss tool cues. Support: 330
Pick Cool then Place in Receptacle	AtomicOp Sequence: go.to → take → go.to → cool → go.to → move Pre-states: target.located, tool.ready, navigation.needed Post-states: process.progress, target.placed Example: cool some mug and put it in coffeemachine. Failure Hint: Search loops without locating the target. Support: 330

Figure 5: Examples of skill representation.

A.4 TYPED EDGE INDUCTION

We construct five types of edges, where each edge is represented as a directed triple (u, v, ℓ) with source node u , target node v , and edge type ℓ . Here, skill nodes are denoted by $s \in V_S$, AtomicOp nodes by $o \in V_O$. For statistical edges, we store a support count and a confidence score to indicate the strength of the construction evidence.

Decomposition Edges. The `decomposes_to` edge is directly derived from each discovered skill node to the AtomicOp nodes in its ordered procedure. For a skill node s with procedure $(o_1, \dots, o_n)$, we add $(s, o_k, \text{decomposes_to})$ and store the order index k . This edge specifies how a high-level skill is decomposed into executable AtomicOp templates.

Temporal Transition Edges. The `can_follow` edge captures temporal continuation patterns in successful trajectories. We construct this relation at two granularities. For skill-to-skill edges, we count how often a skill node appears after another skill node in successful trajectories. For AtomicOp-to-AtomicOp edges, we count adjacent AtomicOp transitions. The confidence is the relative outgoing frequency:

$$\text{conf}(u, v) = \frac{\text{count}(u \rightarrow v)}{\sum_{v'} \text{count}(u \rightarrow v')}. \quad (10)$$

You are given a skill discovered from recurring AtomicOp sequences.
Generate a concise textual representation grounded only in the provided information.

Requirements:

- Use a short and specific skill name.
- Write one concise paragraph describing the procedure and its applicability.
- Summarize one or two failure hints.
- Do not add, remove, or reorder AtomicOps.
- Do not invent arguments, states, or tools.
- Return valid JSON only.

Return JSON with keys:

name, description, failure_modes.

AtomicOp sequence:

{skill.action_signature}

Pre-states:

{skill.prestates}

Post-states:

{skill.poststates}

Argument candidates:

{skill.arg_candidates}

Examples:

{skill.examples}

Figure 6: Prompt template for skill text refinement.

For each source skill node, we keep at most 3 following skill nodes. For each source AtomicOp node, we keep at most 2 following AtomicOp nodes, requiring count ≥ 2 and conf ≥ 0.2 . This edge provides temporally plausible next skill or AtomicOp candidates.

Compatibility Edges. The `compatible_with` edge is constructed only between skill nodes. It is induced from skill co-occurrence in successful trajectories and does not require the two skill nodes to be adjacent. If two skill nodes frequently appear in the same successful task context and their pre/post-states do not conflict, we add symmetric directed edges between them. The confidence is computed as the relative co-occurrence frequency:

$$\text{conf}(s_i, s_j) = \frac{\text{count}(s_i, s_j)}{\sum_{s'} \text{count}(s_i, s')}. \quad (11)$$

We retain at most 4 compatible targets for each source skill node and require the co-occurrence count to be at least 2. This edge indicates which skills can jointly serve the same task or subgoal.

Support Edges. We infer this relation from the local action context of skill nodes in successful trajectories. For each trajectory segment matched to a skill node s , we examine the immediately preceding AtomicOp node o . An AtomicOp node is considered a support candidate only when its post-states overlap with the pre-states of the skill node:

$$\text{conf}(o, s) = J(\text{post}(o), \text{pre}(s)), \quad (12)$$

where $J(\cdot, \cdot)$ denotes Jaccard overlap over state predicates. We accumulate counts over all matched trajectory segments, keep support edges with count at least 2 and confidence at least 0.45, and retain at most 4 support AtomicOp nodes for each skill node. This edge provides auxiliary operations when the current state does not satisfy the prerequisites of a skill node, such as opening a container, selecting an option, or obtaining a required tool.

Recovery Edges. The `recovers_with` edge connects a recovery skill/AtomicOp node to a failed or stalled skill/AtomicOp node. We infer this relation from two sources.

First, we extract direct recovery evidence from failed trajectories. When a trajectory contains a repeated ineffective AtomicOp pattern followed by a different AtomicOp pattern, e.g., o_a, o_a, o_b , we treat o_b as a recovery candidate for o_a . This yields an AtomicOp-level edge $o_b \rightarrow o_a$. If the two

```

You are selecting the active skill node for HiSkill execution.
Choose exactly one skill node from the provided candidates.
Do not create new skill nodes.

Rules:
- For initial selection, rerank the provided top-2 skill nodes.
- If the current skill is finished, select only from candidates connected by can_follow or
  compatible_with.
- If execution is stuck or repeatedly fails, select only from candidates connected by
  recovers_with.
- Return valid JSON only.

Return JSON with keys:
selected_skill_id, confidence, reason.

Task:
{task_description}

Current observation:
{current_observation}

Task state:
{task_state}

Active skill:
{active_skill}

Candidate skill nodes:
{candidate_skill_nodes}

```

Figure 7: Prompt template for initial skill selection and runtime skill switching.

AtomicOp nodes belong to different skill nodes, we also add a skill-level recovery edge between the corresponding skill nodes.

Second, we infer semantic recovery edges from failure evidence associated with failed or stalled source nodes. Here, failure evidence refers to the failure hints, repeated failed action, missing state predicate, or non-progress signal extracted from failed trajectories. For each source node u , we build a recovery query x_u^{rec} from this evidence and retrieve candidate skill/AtomicOp nodes. For each candidate node v , we compute three matching signals.

$$\text{score}_{\text{rec}}(v | u) = 2 \text{Dense}(x_u^{\text{rec}}, d_v) + \text{Sparse}(x_u^{\text{rec}}, d_v) + \text{Sim}_{\text{state}}(u, v), \quad (13)$$

where $\text{Dense}(\cdot)$ and $\text{Sparse}(\cdot)$ follow the definitions in Eq. (5). We assign a larger weight to $\text{Dense}(\cdot)$ because semantic similarity is the primary signal for finding recovery nodes that are relevant to the failure evidence. The state compatibility term is computed as:

$$\text{Sim}_{\text{state}}(u, v) = J(\text{pre}(u), \text{pre}(v)) + J(\text{post}(u), \text{post}(v)). \quad (14)$$

A semantic recovery edge is retained only when the final recovery score is at least 0.20. For each target node, we keep at most 4 incoming recovery edges. This edge is used when execution fails, stagnates, or enters repeated ineffective actions.

The key distinction between `supports` and `recovers_with` lies in their triggering condition. `supports` is used before executing a skill when missing prerequisites are detected, while `recovers_with` is used after failure, stagnation, or repeated ineffective behavior has been observed.

B DETAILS OF TASK-RELEVANT SKILL SUBGRAPH RETRIEVAL

This section provides implementation details of task-relevant skill subgraph retrieval, including seed node retrieval and graph hydration.

B.1 SEED NODE RETRIEVAL

For seed node retrieval, we first convert both the task and graph nodes into textual representations before computing relevance scores. Specifically, the retrieval query x_q is constructed by concatenating the task description and the initial observation. For each graph node v , we construct a retrieval

```

You are selecting the next AtomicOp for HiSkill execution.
Choose one AtomicOp node or null from the provided candidates.
Do not create new AtomicOp nodes.

Rules:
- In the default case, select the first pending AtomicOp of the active skill.
- If prerequisites are missing, select only from candidates connected by supports.
- If execution is stuck or repeatedly fails, select only from candidates connected by
  recovers_with.
- If no suitable AtomicOp exists, generate an action directly.
- Return valid JSON only.

Return JSON with keys:
selected_op_id or null, confidence, reason.

Task:
{task_description}

Current observation:
{current_observation}

Task state:
{task_state}

Active skill:
{active_skill}

Executed AtomicOps:
{executed_atomicops}

Candidate AtomicOp nodes:
{candidate_atomicop_nodes}

```

Figure 8: Prompt template for AtomicOp selection.

text d_v by concatenating multiple node attributes while skipping empty fields, removing duplicated text segments, and compressing redundant spaces. For skill nodes, d_v contains the skill name, description, pre-states, post-states, examples and failure hints. For AtomicOp nodes, d_v contains the normalized action template and argument candidates.

The dense similarity in Eq. (5) is computed using text-embedding-3-large (OpenAI, 2024a), where the query and node texts are encoded into embeddings and ranked by cosine similarity. The sparse score is computed using BM25 (Robertson & Zaragoza, 2009) based on lexical overlap between the query and node text.

Before combining the two retrieval signals, we independently normalize the dense and sparse scores using min-max normalization:

$$\text{norm}(s_v) = \frac{s_v - \min(s)}{\max(s) - \min(s)}, \quad (15)$$

where s_v denotes the raw dense or sparse score of node v , and s denotes the set of raw scores from all candidate nodes under the corresponding retrieval signal. When all candidate nodes have identical scores, we set all normalized scores to 1.0 if the original score is positive and to 0.0 otherwise. This normalization is applied separately to dense and sparse scores to avoid scale differences between embedding similarity and BM25 scores.

For BM25 indexing, all node texts are tokenized using a unified preprocessing procedure. Specifically, text is converted to lowercase and only alphabetic and numeric tokens are retained using regular expression filtering. The index statistics, including term frequency, text frequency, text length, and average text length, are computed over all node texts and used for BM25 scoring.

Finally, skill nodes and AtomicOp nodes are ranked separately according to Eq. (5). The top- K nodes are selected as retrieval seed nodes, and their union forms the seed node set V_{seed} .

```

You are the final HiSkill action grounder; please produce one concrete executable action.

Rules:
- If the target source is AtomicOp, instantiate the normalized action template using the
  provided argument candidates and evidence from the task, current observation, and task
  state.
- Keep the normalized template structure unchanged.
- If the target source is LLM direct-action, AtomicOp instantiation is bypassed.
- Follow the environment action syntax.
- Return valid JSON only.

Return JSON with keys:
action, arguments.

Task:
{task_description}

Current observation:
{current_observation}

Task state:
{task_state}

Selected AtomicOp:
{selected_atomicop_or_none}

Normalized action template:
{normalized_action_template}

Argument candidates:
{argument_candidates}

```

Figure 9: Prompt template for action grounding.

B.2 GRAPH HYDRATION

After seed node retrieval, we expand the retrieved nodes using the typed relations to form the task-relevant subgraph. Specifically, for each selected skill node, we include the AtomicOp nodes connected by `decomposes_to` edges to preserve its executable action templates. For the remaining typed edges, we perform one-hop neighbor expansion by including nodes connected to the selected nodes. Candidate neighbors are ranked using Eq. (5). To control the size of the retrieved subgraph, we keep the top-2 neighbors for each non-decomposition relation. Finally, we retain all typed edges among the selected nodes to form the task-relevant subgraph $G_q = (V_q, E_q)$.

C DETAILS OF SUBGRAPH-GUIDED TASK EXECUTION

This appendix provides implementation details of skill selection and switching, AtomicOp selection, and action grounding.

C.1 SKILL SELECTION AND SWITCHING

Initial Skill Selection. The skill nodes in V_q are ranked according to the retrieval score in Eq. (5). The top-2 skill nodes are provided to the LLM, which reranks them according to the task description, initial observation, and task state, and selects the more suitable node as the initial active skill.

Runtime Skill Switching. The active skill is switched according to the current task state and the typed relations in G_q . When the active skill is finished, successor candidates are collected through `can_follow` or `compatible_with` edges. When execution is stuck or repeatedly fails, recovery candidates are collected through `recovers_with` edges.

The prompt templates for initial skill selection and runtime skill switching are shown in Figure 7.

Dataset	Traj.		Split			Skill Graph				Retrieved Subgraph		
	Succ.	Fail.	Val.	Seen	Unseen	Ops	Skills	Edges	Ops/Skl.	Ops	Skills	Edges
ALFWorld	1262	1545	312	140	134	112	66	649	3.8	11.1	8.1	70.2
WebShop	646	996	182	500	–	134	69	608	2.8	9.2	7.7	66.2
ScienceWorld	800	535	148	194	211	134	67	573	2.7	13.7	9.3	62.4

Table 5: Statistics of datasets.

C.2 ATOMICOP SELECTION

AtomicOp selection is conditioned on the active skill, current task state, and typed relations in G_q . When a skill becomes active, the controller tracks its AtomicOp execution progress according to the order indices of the `decomposes_to` edges, marking each sequence position as executed or pending. By default, the first pending AtomicOp of the active skill is selected. When prerequisites are missing, support candidates are collected through `supports` edges. When execution is stuck or repeatedly fails, recovery candidates are collected through `recovers_with` edges. If no candidate is suitable, the system enables an LLM direct-action.

The prompt template for AtomicOp selection is shown in Figure 8.

C.3 ACTION GROUNDING

When the selected target is an AtomicOp node, the system instantiates its normalized action template. Placeholder values are resolved using the AtomicOp argument candidates together with evidence from the task description, current observation, and task state. The instantiated template is then converted into a concrete action that follows the environment action syntax.

When the selected target is an LLM direct-action, AtomicOp instantiation is bypassed, and the LLM directly generates one concrete action. Figure 9 shows the prompt template.

D DATASET DETAILS

This section provides additional details about the three interactive datasets used in our experiments. Table 5 summarizes the trajectory statistics, evaluation splits, constructed hierarchical skill graph sizes, and average retrieved subgraph sizes.

ALFWorld. ALFWorld (Shridhar et al., 2021) is a text-based embodied environment aligned with the ALFRED benchmark. In each episode, the system receives a textual goal and completes it through multi-turn interaction with the environment using text commands. Following prior work (Feng et al., 2025), ALFWorld contains six common household task categories: Pick and Place (Pick), Examine in Light (Look), Clean and Place (Clean), Heat and Place (Heat), Cool and Place (Cool), and Pick Two and Place (Pick2).

WebShop. WebShop (Yao et al., 2022) is a web-based interactive shopping environment designed to evaluate sequential decision making in realistic online shopping scenarios. Given a user instruction, the system interacts with a shopping website through search, browsing, comparison, option selection, and purchase actions. The environment requires understanding compositional textual constraints and maintaining progress across multiple interaction steps.

ScienceWorld. ScienceWorld (Wang et al., 2022) is a text-based scientific experimentation environment for evaluating grounded scientific reasoning and interactive task completion. The system must observe the environment, locate relevant objects, operate tools or instruments, and use environment feedback to complete science-oriented tasks. The environment emphasizes long-horizon decision making, state tracking, and procedural control under partial observations.

Method	ALFWorld													
	Seen							Unseen						
	Pick	Look	Clean	Heat	Cool	Pick2	All	Pick	Look	Clean	Heat	Cool	Pick2	All
<i>Prompt-based Methods</i>														
ReAct	94.29	92.31	74.07	43.75	56.00	79.17	75.00	95.83	94.44	87.10	47.83	76.19	82.35	80.60
Reflexion	<u>97.14</u>	69.23	<u>96.30</u>	<u>87.50</u>	76.00	75.00	85.71	100.00	83.33	80.65	<u>73.91</u>	90.48	58.82	82.09
<i>Memory-based Methods</i>														
ExpeL	91.43	92.31	96.30	68.75	60.00	79.17	82.14	79.17	77.78	87.10	65.22	57.14	64.71	73.13
Mem0	<u>97.14</u>	92.31	92.59	<u>87.50</u>	76.00	95.83	90.71	100.00	66.67	90.32	56.52	95.24	82.35	82.84
MemP	<u>97.14</u>	100.00	100.00	<u>87.50</u>	<u>92.00</u>	91.67	95.00	100.00	100.00	93.55	82.61	100.00	76.47	92.54
SimpleMem	<u>97.14</u>	100.00	62.96	43.75	68.00	87.50	77.86	95.83	83.33	80.65	26.09	52.38	70.59	68.66
<i>Skill-based Methods</i>														
Vector Skills	<u>97.14</u>	84.62	77.78	56.25	56.00	83.33	77.86	100.00	88.89	77.42	47.83	61.90	82.35	76.12
SkillNet	<u>97.14</u>	76.92	88.89	93.75	88.00	100.00	92.14	91.67	72.22	93.55	65.22	90.48	100.00	85.82
GoS	94.29	92.31	81.48	31.25	40.00	62.50	69.29	87.50	88.89	77.42	34.78	52.38	70.59	68.66
HiSkill	100.00	100.00	92.59	75.00	100.00	<u>95.83</u>	95.00	100.00	100.00	90.32	69.57	100.00	100.00	92.54

Table 6: Performance on ALFWorld with GPT-5.2-Codex: Success rate (%) is reported. The best results are highlighted in boldface, and the second-best results are underlined.

Statistics. Table 5 reports the statistics used in our experiments. The Success and Failure columns denote trajectories used for hierarchical skill graph construction after holding out the validation split. The Val. column denotes the 10% validation split held out from training data. The Skill Graph columns report the number of AtomicOp nodes, skill nodes, typed edges, and the average number of AtomicOps per skill. The Retrieved Subgraph columns report the average number of AtomicOps, skills, and edges in task-relevant retrieved skill subgraphs.

E ADDITIONAL EXPERIMENTAL RESULTS

E.1 MAIN RESULTS ON GPT-5.2-CODEX

Tables 6 and 7 report the results with GPT-5.2-Codex. The overall trend is consistent with the Gemini-2.5-Pro results: our method remains the strongest or tied-strongest approach across the three environments, showing that the gains of HiSkill are consistent across different backbone LLMs. Compared with the strongest baseline across all datasets and splits, our method achieves an average relative improvement of 11.91% in success rate and 18.31% in score. On ALFWorld, our method matches the best baseline on both Seen and Unseen overall success rates, where performance is already close to saturated. On WebShop, compared with the strongest baseline, our method achieves relative improvements of 26.64% in score and 34.98% in success rate. On ScienceWorld, the relative improvements over the strongest baseline are 18.91% / 19.00% on Seen and 11.15% / 21.28% on Unseen in score / success rate. These results confirm that explicitly organizing skill nodes, AtomicOp nodes, and typed relations is beneficial beyond a single backbone, especially in environments requiring fine-grained action grounding and long-horizon progress control.

E.2 SCORE-BASED HYPERPARAMETER ANALYSIS

Figure 10 reports the validation score under different retrieval weights λ and seed node budgets K on WebShop and ScienceWorld. ALFWorld is omitted because it only provides a binary success signal. The score trends are consistent with the success rate analysis in Figure 3: WebShop favors a balanced dense-sparse setting around $\lambda = 0.5$, while ScienceWorld benefits more from semantic retrieval around $\lambda = 0.75$. For the seed node budget, $K = 6$ achieves the best ScienceWorld score and remains competitive on WebShop, supporting its use as the default budget in the main experiments.

Method	WebShop		ScienceWorld			
	Score	Succ.	Seen		Unseen	
			Score	Succ.	Score	Succ.
<i>Prompt-based Methods</i>						
ReAct	54.11	42.40	68.03	63.92	58.79	54.98
Reflexion	51.75	<u>48.60</u>	66.55	59.28	54.18	45.97
<i>Memory-based Methods</i>						
ExpeL	35.71	31.40	60.34	54.12	64.02	53.55
Mem0	57.83	46.20	73.50	69.07	67.46	64.93
MemP	<u>59.24</u>	48.00	<u>77.21</u>	<u>73.20</u>	<u>75.43</u>	<u>66.82</u>
SimpleMem	48.62	38.00	64.41	67.53	55.55	58.29
<i>Skill-based Methods</i>						
Vector Skills	58.11	46.80	64.45	63.40	55.07	55.45
SkillNet	27.76	23.60	65.32	64.43	56.64	60.19
GoS	56.21	48.20	70.26	67.01	63.06	61.14
HiSkill	75.02	65.60	91.81	87.11	83.84	81.04

Table 7: Performance on WebShop and ScienceWorld with GPT-5.2-Codex: Average score and success rate (%) are reported. The best results are highlighted in boldface, and the second-best results are underlined.

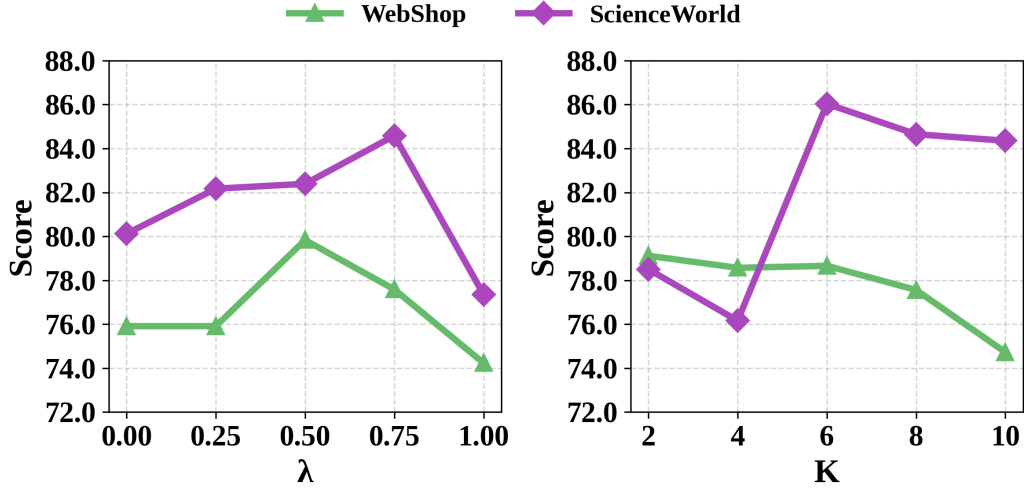


Figure 10: Validation sensitivity to the retrieval weight λ and seed node budget K measured by task score.

E.3 CASE STUDY

We present a WebShop case to illustrate how HiSkill retrieves a task-relevant subgraph, monitors a task state and an active skill, and leverages the retrieved subgraph with an LLM for skill switching, AtomicOp selection, and action grounding.

Task and Initial Context

Task q Find me wash cold, machine wash men's shirts, classic fit with color: black, and fit type: youth, and size: 4t, and price lower than 50.00 dollars.
Observation o_0 Search page.

Task State z_0 :

```
stage: search
current_product: none
selected_options: []
search_history: []
visited_products: []
rejected_products: []
visited_navigation: []
recent_actions: []
recent_failures: []
missing_constraints: [product, color=black, fit_type=youth, size=4t]
```

Skill Subgraph Retrieval

Seed Skill Nodes

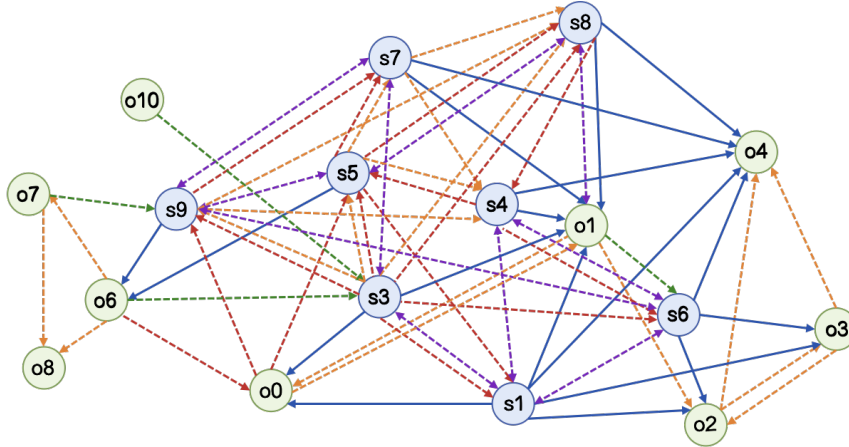
1. s3: SearchApparelWithConstraints
2. s4: FilterAndBuyApparelByAttributes
3. s6: SelectAndBuyApparelVariant
4. s9: ApparelVariantSelectAndBuyNow
5. s7: SelectSizeColorAndBuyNow
6. s8: ApparelColorSizeBuy

Seed AtomicOp Nodes

1. o10: inspect
2. o7: pick
3. o6: click_product()
4. o2: select_color()
5. o3: select_size()
6. o4: buy()

Expanded nodes: s1: ApparelSearchSelectBuy, s5: SelectProductASIN, o0: search(), o1: click(), o8: submit.
 After graph hydration, the task-relevant subgraph contains 8 skill nodes, 9 AtomicOp nodes, and 66 typed edges.

Task-relevant Subgraph G_q



Step 0

Active Skill Node

s1

Subgraph Decision

Select the initial skill node and follow s1 $\xrightarrow{\text{decomposes_to}}$ o0.

Selected Target

o0: search(query=men cold machine wash shirts black)

Grounded Action

search[men cold machine wash shirts black]

Observation o_1

A search-result page with 50 products is returned. Candidate B09QQP3356 is visible at \$10.99.

Task State z_1 :

```
stage: result
current_product: none
selected_options: []
search_history: [men cold machine wash shirts black]
visited_products: []
rejected_products: []
visited_navigation: []
recent_actions: [search[men cold machine wash shirts black]]
recent_failures: []
missing_constraints: [product, color=black, fit_type=youth, size=4t]
```

Step 1

Active Skill Node s1
Subgraph Decision Continue the active skill node through s1 $\xrightarrow{\text{decomposes.to}}$ o1.
Selected Target o1: click(asin=B09QQP3356)
Grounded Action click[b09qqp3356]
Observation o2 The product-detail page is opened. Black is available, but size 4t and fit type youth are absent.
Task State z2:
stage: detail
current_product: B09QQP3356
selected_options: []
search_history: [men cold machine wash shirts black]
visited_products: [B09QQP3356]
rejected_products: []
visited_navigation: []
recent_actions: [search[men cold machine wash shirts black], click[b09qqp3356]]
recent_failures: [candidate_missing_size=4t, candidate_missing_fit_type=youth]
missing_constraints: [fit_type=youth, size=4t]

Step 2

Active Skill Node s3
Subgraph Decision Execution stuck, switch to a recovery skill node through: s3 $\xrightarrow{\text{recovers.with}}$ s1. Then select s3 $\xrightarrow{\text{decomposes.to}}$ o1.
Selected Target o1: click(target=back to search)
Grounded Action click[back to search]
Observation o3 The search page is displayed again.
Task State z3:
stage: search
current_product: none
selected_options: []
search_history: [men cold machine wash shirts black]
visited_products: [B09QQP3356]
rejected_products: [B09QQP3356 -> {missing_size=4t, missing_fit_type=youth}]
visited_navigation: [back to search]
recent_actions: [search[men cold machine wash shirts black], click[b09qqp3356], click[back to search]]
recent_failures: [candidate_missing_size=4t, candidate_missing_fit_type=youth]
missing_constraints: [product, fit_type=youth, size=4t]

Step 3

Active Skill Node s3
Subgraph Decision Continue the active skill node through s3 $\xrightarrow{\text{decomposes.to}}$ o0.
Selected Target o0: search(query=men shirts black 4t)
Grounded Action search[men shirts black 4t]
Observation o4 A refined result page is returned. Candidate B09P39QN2W appears at \$16.99.
Task State z4:
stage: result
current_product: none
search_history: [men cold machine wash shirts black, men shirts black 4t]
visited_products: [B09QQP3356]
rejected_products: [B09QQP3356 -> {missing_size=4t, missing_fit_type=youth}]
visited_navigation: [back to search]
recent_actions: [search[men cold machine wash shirts black], click[b09qqp3356], click[back to search], search[men shirts black 4t]]
recent_failures: []
missing_constraints: [product, color=black, fit_type=youth, size=4t]

Step 4

Active Skill Node s9
Subgraph Decision Select a successor skill node through s3 $\xrightarrow{\text{can.follow}}$ s9. Then select s9 $\xrightarrow{\text{decomposes.to}}$ o6.
Selected Target o6: click.product(asin=B09P39QN2W)
Grounded Action click[b09p39qn2w]
Observation o5 The new product provides youth, black, and 4t options, costs \$16.99, and exposes Buy Now.
Task State z5:
stage: detail
current_product: B09P39QN2W
selected_options: []
search_history: [men cold machine wash shirts black, men shirts black 4t]
visited_products: [B09QQP3356, B09P39QN2W]
rejected_products: [B09QQP3356 -> {missing_size=4t, missing_fit_type=youth}]
visited_navigation: [back to search]
recent_actions: [click[b09qqp3356], click[back to search], search[men shirts black 4t], click[b09p39qn2w]]
recent_failures: []
missing_constraints: [color=black, fit_type=youth, size=4t]

Step 5

Active Skill Node s6
Subgraph Decision Select a successor skill node through s9 $\xleftarrow{\text{compatible.with}}$ s6. Then select s6 $\xrightarrow{\text{decomposes.to}}$ o2.
Selected Target o2: select_color(value=black)
Grounded Action click[black]
Observation o6 Color black is available and selected.
Task State z6:
stage: detail
current_product: B09P39QN2W
selected_options: [black]
search_history: [men cold machine wash shirts black, men shirts black 4t]
visited_products: [B09QQP3356, B09P39QN2W]

```
rejected_products: [B09QQP3356 -> {missing_size=4t, missing_fit_type=youth}]
visited_navigation: [back to search]
recent_actions: [click[back to search], search[men shirts black 4t], click[b09p39qn2w], click[black]]
recent_failures: []
missing_constraints: [fit_type=youth, size=4t]
```

Step 6

Active Skill Node s6
Subgraph Decision Continue the active skill node through s6 $\xrightarrow{\text{decomposes.to}}$ o3.
Selected Target o3: select_size(value=4t)
Grounded Action click[4t]
Observation o₇ Size 4t is selected.
Task State z₇:
stage: detail
current_product: B09P39QN2W
selected_options: [black, 4t]
search_history: [men cold machine wash shirts black, men shirts black 4t]
visited_products: [B09QQP3356, B09P39QN2W]
rejected_products: [B09QQP3356 -> {missing_size=4t, missing_fit_type=youth}]
visited_navigation: [back to search]
recent_actions: [search[men shirts black 4t], click[b09p39qn2w], click[black], click[4t]]
recent_failures: []
missing_constraints: [fit_type=youth]

Step 7

Active Skill Node s6
Subgraph Decision The missing fit constraint, select the support AtomicOp node through o1 $\xrightarrow{\text{supports}}$ s6.
Selected Target o1: click(value=youth)
Grounded Action click[youth]
Observation o₈ Fit type youth is selected. All required options are satisfied, and Buy Now remains available.
Task State z₈:
stage: detail
current_product: B09P39QN2W
selected_options: [black, 4t, youth]
search_history: [men cold machine wash shirts black, men shirts black 4t]
visited_products: [B09QQP3356, B09P39QN2W]
rejected_products: [B09QQP3356 -> {missing_size=4t, missing_fit_type=youth}]
visited_navigation: [back to search]
recent_actions: [click[b09p39qn2w], click[black], click[4t], click[youth]]
recent_failures: []
missing_constraints: []

Step 8

Active Skill Node s6
Subgraph Decision All constraints are satisfied, and continue the active skill node through s6 $\xrightarrow{\text{decomposes.to}}$ o4.
Selected Target o4: buy(asin=B09P39QN2W, size=4t, color=black, fit_type=youth)
Grounded Action click[buy now]
Observation o₉ The purchase succeeds and the page reports a score of 100.0.
Task State z₉:
stage: done
current_product: B09P39QN2W
selected_options: [black, 4t, youth]
search_history: [men cold machine wash shirts black, men shirts black 4t]
visited_products: [B09QQP3356, B09P39QN2W]
rejected_products: [B09QQP3356 -> {missing_size=4t, missing_fit_type=youth}]
visited_navigation: [back to search]
recent_actions: [click[black], click[4t], click[youth], click[buy now]]
recent_failures: []
missing_constraints: []